

DevOps & Git – Practical Exam Guide

All 10 Experiments – Step by Step

Experiment 1: Initialize a Git Repository

Goal: Create a local Git repository, add a file, and commit using Git commands.

Prerequisites

- Install Git from <https://git-scm.com>
- Open Terminal (Linux/Mac) or Git Bash (Windows)

Step-by-Step Commands

Step 1 – Create a project folder:

```
mkdir myproject  
cd myproject
```

Step 2 – Initialize the Git repository:

```
git init
```

 *Note: This creates a hidden .git folder that tracks all changes.*

Step 3 – Create a text file:

```
echo "Hello, Git!" > hello.txt
```

Step 4 – Check the status:

```
git status
```

 *Note: You will see hello.txt listed as an untracked file.*

Step 5 – Stage the file (add to index):

```
git add hello.txt
```

Or to add all files at once:

```
git add .
```

Step 6 – Commit the file:

```
git commit -m "Initial commit: added hello.txt"
```

Step 7 – View commit history:

```
git log
```

Expected Output

- `git init` → Initialized empty Git repository
 - `git status` → shows `hello.txt` as new file
 - `git log` → shows your first commit with message
-

Experiment 2: Push Local Project to GitHub

Goal: Create a GitHub repository and push your local project to it.

Prerequisites

- GitHub account created at github.com
- Git installed and configured with your name and email

Step-by-Step

Step 1 – Configure Git with your identity (do this once):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Step 2 – Create a new repository on GitHub:

- Go to github.com → click + → New repository
- Give it a name e.g. `myproject`
- Do NOT check 'Initialize this repository' (we already have local files)
- Click Create repository

Step 3 – Link local repo to GitHub (copy the URL from GitHub):

```
git remote add origin https://github.com/yourusername/myproject.git
```

Step 4 – Rename the default branch to main:

```
git branch -M main
```

Step 5 – Push your code to GitHub:

```
git push -u origin main
```

 *Note: -u sets upstream so future pushes just need: `git push`*

Step 6 – Verify on GitHub by refreshing the page. Your file should appear.

Key Commands Summary

- `git remote add origin <URL>` – connects local to remote
- `git push -u origin main` – pushes and sets tracking

Experiment 3: Branches and Merging in Git

Goal: Create two branches, modify a file in each, then merge into main.

Step-by-Step

Step 1 – Start from main branch:

```
git checkout main
```

Step 2 – Create and switch to branch1:

```
git checkout -b branch1
```

 *Note: git checkout -b creates a new branch AND switches to it.*

Step 3 – Modify the file in branch1:

```
echo "Change from branch1" >> hello.txt
git add hello.txt
git commit -m "Changes from branch1"
```

Step 4 – Switch back to main and create branch2:

```
git checkout main
git checkout -b branch2
```

Step 5 – Modify the file in branch2:

```
echo "Change from branch2" >> hello.txt
git add hello.txt
git commit -m "Changes from branch2"
```

Step 6 – Merge branch1 into main:

```
git checkout main
git merge branch1
```

Step 7 – Merge branch2 into main:

```
git merge branch2
```

Step 8 – View final file:

```
cat hello.txt
```

Key Concepts

- Branch – an independent line of development
- Merge – combining changes from two branches into one

- HEAD – points to the currently active branch

Experiment 4: Pull Request and Merge Conflict

Goal: Create a pull request on GitHub and resolve a merge conflict.

Part A – Create a Pull Request

Step 1 – Create a new branch and push it:

```
git checkout -b feature-branch
echo "Feature work" >> hello.txt
git add . && git commit -m 'Feature update'
git push origin feature-branch
```

Step 2 – On GitHub:

- Go to your repo → you will see 'Compare & pull request' button
- Click it → add a title and description
- Click Create pull request
- Click Merge pull request → Confirm merge

Part B – Create and Resolve a Merge Conflict

Step 1 – On main, edit the file:

```
git checkout main
echo "Main branch line" > conflict.txt
git add . && git commit -m 'Main edit'
```

Step 2 – On a new branch, edit the same file differently:

```
git checkout -b conflict-branch
echo "Conflict branch line" > conflict.txt
git add . && git commit -m 'Conflict branch edit'
```

Step 3 – Switch to main and merge (conflict will occur):

```
git checkout main
git merge conflict-branch
```

Step 4 – Open conflict.txt, you will see:

```
<<<<<<< HEAD
Main branch line
=====
Conflict branch line
>>>>>>> conflict-branch
```

Step 5 – Manually edit to keep what you want, then:

```
git add conflict.txt
git commit -m "Resolved merge conflict"
```

 *Note: Always remove the <<<, ===, >>> markers when resolving conflicts.*

Experiment 5: GitHub Actions – Build Success Workflow

Goal: Configure a GitHub Actions workflow that prints a success message on every push.

Step-by-Step

Step 1 – In your repo, create the workflow directory:

```
mkdir -p .github/workflows
```

Step 2 – Create the workflow file:

```
nano .github/workflows/hello.yml
```

Step 3 – Paste this content into the file:

```
name: Hello World Workflow

on:
  push:
    branches: [ main ]

jobs:
  greet:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Print Success Message
        run: echo 'Build Successful! Workflow triggered on push.'
```

Step 4 – Save, commit, and push:

```
git add .github/workflows/hello.yml
git commit -m "Add GitHub Actions workflow"
git push origin main
```

Step 5 – On GitHub, go to Actions tab to see the workflow running.

Key YAML Terms

- on: push – trigger when code is pushed
- runs-on – the OS of the runner machine (ubuntu-latest)
- steps – individual tasks inside a job
- run – shell command to execute

Experiment 6: CI/CD Workflow – Build and Deploy HTML Page

Goal: Use GitHub Actions to build and deploy a static HTML page automatically.

Step-by-Step

Step 1 – Create a simple HTML file in your repo:

```
nano index.html
```

Paste this:

```
<!DOCTYPE html>
<html>
  <body><h1>Deployed via GitHub Actions!</h1></body>
</html>
```

Step 2 – Create the CI/CD workflow file:

```
nano .github/workflows/deploy.yml
```

Paste this:

```
name: Deploy Static Page

on:
  push:
    branches: [ main ]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Deploy to GitHub Pages
        uses: peaceiris/actions-gh-pages@v3
        with:
          github_token: ${ secrets.GITHUB_TOKEN }
          publish_dir: ./
```

Step 3 – Enable GitHub Pages:

- Go to repo → Settings → Pages
- Under Source, select gh-pages branch
- Click Save

Step 4 – Push the files:

```
git add .
git commit -m "Add CI/CD deploy workflow"
git push origin main
```

Step 5 – After the workflow runs, visit:

<https://yourusername.github.io/myproject/>

 *Note: GITHUB_TOKEN is automatically provided by GitHub – no manual setup needed.*

Experiment 7: Automated Testing with GitHub Actions

Goal: Implement automated testing in GitHub Actions using Python or Node.js test files.

Option A – Python Testing

Step 1 – Create a Python function file (math_ops.py):

```
def add(a, b):
    return a + b
```

Step 2 – Create a test file (test_math.py):

```
from math_ops import add

def test_add():
    assert add(2, 3) == 5
    assert add(0, 0) == 0
    print('All tests passed!')

test_add()
```

Step 3 – Create the workflow file (.github/workflows/test.yml):

```
name: Python Tests

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
```

```
- uses: actions/setup-python@v4
  with:
    python-version: '3.x'
- name: Run tests
  run: python test_math.py
```

Option B – Node.js Testing

Step 1 – Create math.js:

```
function add(a, b) { return a + b; }
module.exports = { add };
```

Step 2 – Create test_math.js:

```
const { add } = require('./math');
console.assert(add(2,3) === 5, 'Test failed');
console.log('All Node.js tests passed!');
```

Step 3 – Workflow file:

```
name: Node.js Tests
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: '18'
      - run: node test_math.js
```

Step 4 – Push everything. Check Actions tab for test results.

Experiment 8: Dockerfile – Build a Docker Image

Goal: Create a Dockerfile for a simple web application and build a Docker image.

Prerequisites

- Docker installed: <https://docs.docker.com/get-docker/>
- Verify: `docker --version`

Step-by-Step

Step 1 – Create your app file (app.py):

```
from flask import Flask
```



```
app = Flask(__name__)

@app.route('/')
def home():
    return 'Hello from Docker!'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Step 2 – Create requirements.txt:

```
flask
```

Step 3 – Create the Dockerfile (no extension):

```
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

Step 4 – Build the Docker image:

```
docker build -t myflaskapp .
```

 *Note: The dot (.) means 'use the Dockerfile in the current directory'.*

Step 5 – Run the container:

```
docker run -p 5000:5000 myflaskapp
```

Step 6 – Open browser and visit: <http://localhost:5000>

Key Dockerfile Instructions

- FROM – base image to start from
- WORKDIR – sets working directory inside container
- COPY – copies files from host to container
- RUN – executes commands during build
- EXPOSE – documents which port the app uses
- CMD – default command to run when container starts

Experiment 9: Docker Compose – Web Server + Database

Goal: Use Docker Compose to run two containers: a web server (Nginx) and a database (MySQL).

Prerequisites

- Docker and Docker Compose installed
- Verify: docker compose version

Step-by-Step

Step 1 – Create a project folder:

```
mkdir compose-demo && cd compose-demo
```

Step 2 – Create docker-compose.yml:

```
version: '3.8'

services:
  web:
    image: nginx:latest
    ports:
      - '8080:80'
    depends_on:
      - db

  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: mydb
    ports:
      - '3306:3306'
```

Step 3 – Start both containers:

```
docker compose up -d
```

 **Note:** *-d runs containers in the background (detached mode).*

Step 4 – Verify containers are running:

```
docker compose ps
```

Step 5 – Visit the Nginx web server:

```
http://localhost:8080
```

Step 6 – Stop all containers:

```
docker compose down
```

Key Docker Compose Concepts

- services – defines each container
- image – which Docker image to use
- ports – HOST:CONTAINER port mapping
- environment – environment variables passed into the container
- depends_on – ensures db starts before web

Experiment 10: Ansible Playbook – Install Apache/Nginx

Goal: Write and execute an Ansible playbook to install Apache or Nginx on a Linux machine.

Prerequisites

- Ansible installed on control machine: `sudo apt install ansible -y`
- Target Linux machine (can be localhost for practice)

Step-by-Step – Install Nginx

Step 1 – Check Ansible is installed:

```
ansible --version
```

Step 2 – Set up inventory file (hosts.ini):

```
[webservers]
localhost ansible_connection=local
```

 *Note: Using `ansible_connection=local` means it runs on your own machine, no SSH needed.*

Step 3 – Create the playbook file (install_nginx.yml):

```
---
- name: Install and Start Nginx
  hosts: webservers
  become: yes

  tasks:
    - name: Update apt cache
      apt:
        update_cache: yes

    - name: Install Nginx
      apt:
        name: nginx
```

```
state: present

- name: Start Nginx service
  service:
    name: nginx
    state: started
    enabled: yes
```

Step 4 – Run the playbook:

```
ansible-playbook -i hosts.ini install_nginx.yml
```

Step 5 – Verify Nginx is running:

```
systemctl status nginx
```

Step 6 – Open browser and visit: <http://localhost>

For Apache Instead of Nginx

Simply replace nginx with apache2 in the playbook:

```
name: apache2
```

Key Ansible Terms

- Playbook – a YAML file describing automation tasks
- hosts – which machines to run tasks on
- become: yes – run as sudo/root
- tasks – list of actions to perform
- apt module – manages packages on Ubuntu/Debian
- service module – starts/stops/enables services

Quick Reference – All Key Commands

Experiment	Key Command	Purpose
Exp 1 – Git Init	<code>git init / git add / git commit</code>	Start a local git repo
Exp 2 – GitHub Push	<code>git remote add origin / git push</code>	Push to remote GitHub repo
Exp 3 – Branches	<code>git checkout -b / git merge</code>	Create branches and merge
Exp 4 – Pull Request	<code>git push + PR on GitHub</code>	Raise PR and resolve conflicts
Exp 5 – Actions Basics	<code>on: push + echo 'success'</code>	Auto-run on every push

Experiment	Key Command	Purpose
Exp 6 – CI/CD Deploy	<code>actions-gh-pages workflow</code>	Deploy HTML to GitHub Pages
Exp 7 – Testing	<code>python test.py / node test.js</code>	Run tests in Actions pipeline
Exp 8 – Dockerfile	<code>docker build / docker run</code>	Build and run Docker image
Exp 9 – Compose	<code>docker compose up -d</code>	Run multi-container app
Exp 10 – Ansible	<code>ansible-playbook install.yml</code>	Install and start service

All the best for your practical exam!